

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我之前没有正式的多人协同开发经历

思考一下，Git 为什么要设计“暂存-提交”两个步骤？

在日常开发中，我们可能同时修改了多个文件，暂存功能允许我们只挑选本次相关的一部分修改进行提交，保证每一次commit都是一次独立、完整、逻辑清晰的修改，这样提交历史更清晰，回退代码也更安全。并且在正式提交前，我们可以再次审查自己到底暂存了哪些内容，防止把未完成的代码提交到版本库。

git branch 和 git branch -a 的区别是什么？查阅资料并回答。

git branch：用于列出当前本地仓库中的所有本地分支。当前所在的分支前面会有一个 * 号标记。如果加上具体分支名（如 git branch dev），则代表创建名为 dev 的新分支。

git branch -a：其中的 -a 是 --all 的缩写。该命令用于列出所有分支，包括本地分支和远程分支。

3. 在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。

Commit Message 规范：

这篇长文（阮一峰的《Commit message 和 Change log 编写指南》）主要讲了以下三点：

1. 为什么要规范写提交说明（Commit message）

平时写完代码敲 git commit -m “改了” 也能提交，但如果团队几十号人都这样写，代码历史就会变成一锅粥（全是“改了”、“修了一下”）。规范的提交信息要求你写成 type(scope): subject 的格式，它规定了几个常用的类型前缀，如 feat（新功能）、fix（修bug）、docs（改文档）等。

2. 一条完整的提交说明长什么样？

包含三部分：Header（必须，指明类型和简述）、Body（可选，详细描述为什么改，以前什么样现在什么样）、Footer（可选，比如标注“不兼容变动”或者是“关闭了某个Issue”）。

3. 怎么让小白也能写出规范的提交？（工具推荐）

因为手写容易忘格式，文章推荐了工具：commitizen、validate-commit-msg、conventional-changelog

Git Flow 分支控制：这篇文档主要讲了一件事：团队开发时，如何通过不同的分支来分工合作，防止代码乱套，分别为master, develop, feature, release, hotfix这5种核心分支

为什么要学习git：

1. 解决多人协作冲突，建立清晰的分工机制
2. 保证线上代码的绝对安全与稳定性
3. 为自动化部署和持续交付打下基础
4. 降低维护成本，让历史有迹可循

资源管理器

新建文件夹 (2)

xiu

.github

main.c4

Makefile

README.md

main.cpp

tempCodeRunnerFile.c

tempCodeRunnerFile.exe

Git Graph

Untitled-1

main.c4

xiu > C main.c > main()

1#include <stdio.h>

2

3int main()

4{

5// @TODO: print a sentence you want.

6printf("Hello, my name is huangjingxiu!\n");

7//finish my homework

采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更

8<<<<<< HEAD (当前更改)

9printf("i am main.2");

10=====

11printf("i am feature.2");

12>>>>>> feature (传入的更改)

13

14}

问题4

输出

调试控制台

终端

端口

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu> git commit -m "feat: feature再次修改"

[feature baa13d2] feat: feature再次修改

1 file changed, 1 insertion(+), 1 deletion(-)

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu> git checkout main

Switched to branch 'main'

Your branch is ahead of 'origin/main' by 4 commits.

(use "git push" to publish your local commits)

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu> git merge feature

Auto-merging main.c

CONFLICT (content): Merge conflict in main.c

Automatic merge failed; fix conflicts and then commit the result.

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu>

1#include <stdio.h>

2

3int main()

4{

5// @TODO: print a sentence you want.

6printf("Hello, my name is huangjingxiu!\n");

7//finish my homework

8printf("i am main.2");

9

10}

问题

输出

调试控制台

终端

端口

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu> git checkout main

Switched to branch 'main'

Your branch is ahead of 'origin/main' by 4 commits.

(use "git push" to publish your local commits)

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu> git merge feature

Auto-merging main.c

CONFLICT (content): Merge conflict in main.c

Automatic merge failed; fix conflicts and then commit the result.

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu> git add main.c

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu> git commit -m "merge: 解决合并冲突"

[main ac2902c] merge: 解决合并冲突

PS C:\Users\修\Desktop\新建文件夹 (2)\xiu>